

App walkthrough — what to say, tab by tab

One page you can read once and then explain the demo app yourself, without opening any code. Run it with `streamlit run app/demo_app.py` (from the `ml_gpu` env, project root).

What this app is, in one breath

A Streamlit app that walks through an LSM (magnetometer) pipeline-integrity pipeline end to end — raw sensor signal in, a risk-ranked dig list out — using **real trained models on real (synthetic) surveys**, not mocked output. It is a *consumer* of the pipeline's real batch-scoring output, not the serving layer itself (the serving layer is `lsm train / lsm predict`, which write to a database and a GeoJSON export the app then reads).

Two modes, same underlying code path: - **demo** — reads precomputed results straight off disk (`serving/`). Zero compute, cannot fail. This is what you run in front of people. - **live** — re-runs the actual `validate → features → score` pipeline on the spot (~1s), against a real session-private database. Proves it isn't smoke and mirrors. Toggle to this once, live, as the "watch it actually work" moment.

Landing page

Title + tagline + the ROSEN field photo, gated behind a "Launch the demo" button so nobody lands mid-tab on a shared screen. Nothing to explain here beyond "click Launch."

The controls above the tabs

- **Mode:** demo / live, as above.
- **Scenario:** 3 **clean** surveys (passed all validation checks — this is about data quality, *not* whether the model did well) + 1 **corrupted** survey (one sensor reading deliberately pushed out of physical range, to demo tab 6). All three clean scenarios are real surveys the trained model genuinely scored, not fabricated for the demo.
- A **survey** = one pass of the magnetometer sled along a stretch of buried pipe. The same physical line gets surveyed repeatedly over time (like a baseline inspection plus follow-ups), which is how growth/change gets tracked.

Tab 1 — How it works

A workflow diagram of the six real pipeline stages (Ingest → Validate → Features → Detect → Severity+Classify → Risk rank), each one labeled with the actual Python module behind it — so if someone asks "where does X happen," you can point at the box and name the file. Includes an expander listing all **14 real data-quality checks** the validator runs (schema, range, saturation, GPS jump, survey-overlap cross-correlation, etc.).

Say: "Every scenario in this app runs this exact same six-stage path — this tab is the map of it."

Tab 2 — Model performance

This is the tab that answers "where's the baseline, where's IsolationForest, where's the boosting model." It renders the real, auto-generated model card for whatever pipeline is currently loaded — the same numbers `lsm train` prints to the console, not a re-derived summary. Four real comparisons, each with a bootstrap confidence interval, not a point estimate:

Stage	Model	vs. baseline	Real result
3 — Detection	IsolationForest	MAD (median-absolute-deviation) threshold	Recall gap -0.006 [-0.067, 0.050] — statistically indistinguishable. Gate (≥ 0.15 gap) did not pass . IsolationForest <i>does</i> reject interference significantly better, though (CI excludes zero).
4 — Severity	LightGBM quantile regression + split conformal	Global-mean baseline	Coverage 0.920 vs 0.887 target band [0.87, 0.93] — gate passed . MAE 7.46 vs 15.09.
5 — Classification	LightGBM multiclass + isotonic calibration	Majority-class baseline	SCC recall 0.125 — gate did not pass . Interference recall/precision (0.778 / 0.909) is strong — see caveat below.
8 — Growth	Partially-pooled log-linear	"No growth" baseline	Gate passed — model MAE far below baseline's.

Say: "We don't just report the fancy model's number — every model here is compared against a stated, honest baseline, and when the fancy model *doesn't* clear the bar, that's reported as-is, not hidden. IsolationForest vs. MAD is a real example: recall is a tie, but IsolationForest genuinely wastes less dig budget on interference, and we can show why."

Known caveat, worth knowing before someone asks: the four defect subtypes (SCC, weld, dent, corrosion) are assigned **at random** in the synthetic data generator and carry no distinguishing physical signal — so the classifier can't tell them apart (that's exactly why SCC/weld recall sit near the 25% random-guess floor). It genuinely *does* separate **defect vs. interference** well (that distinction *is* physically real in the generator). If asked, say this plainly rather than over-claiming subtype accuracy — full detail in `.claude/skills/lsm-integrity/references/interview-drills.md` under "How do you decide defect type."

Tab 3 — Raw signal

The raw ~45,000 nT field, bx/by/bz as three separate lines — the defect is genuinely invisible here. A log-scale "deviation from median" toggle is the first place it becomes visible. Dashed markers show true defect/interference locations for reference.

Say: "This is the point of the whole pipeline — you cannot see the defect in the raw signal. Everything downstream exists to make it visible."

Tab 4 — Detrend + gradient

Same signal after background removal (two-stage detrend: robust polynomial + rolling-median high-pass). Now both true defects *and* true interference sources show up — separating the two is the model's actual job, not detecting a bump exists.

Tab 5 — Ranked indications

The output an inspection engineer actually reads: a map, a severity/type scatter (with 90% confidence intervals), a table, and a ranked bar chart — same dig-budget-limited set of indications shown four ways. Ranked by `risk_score` (calibrated $P(\text{defect}) \times \text{severity} \times \text{consequence proxy}$) once a classify model is available, falling back to `anomaly_score` otherwise. The dig-budget control (3/5/10 per km) is a pure client-side re-slice, nothing recomputes.

Say: "This is the answer to 'what do I dig first' — not a probability nobody can act on, a ranked list with a stated budget."

Tab 6 — Corrupted survey

Loads the deliberately-broken scenario and shows the validator's actual refusal: which of the 14 checks failed, on which rows, with what value. This is the tab that proves the system knows the difference between "the model is uncertain" and "the data is bad" — most demos never show this half.

Say: "Watch it refuse to score bad data instead of silently producing a confident, wrong answer."

Footer

`pipeline_version` / `feature_version` / `schema_version` / `config_sha256` / `git_sha` — full provenance of exactly what produced whatever's on screen. Point at it if anyone asks "how do you know what model made this prediction."

If you only have 90 seconds

1. Launch → tab 3 (defect is invisible) → tab 4 (now it isn't) → tab 5 (ranked list).
2. Flip Mode to **live** on one clean scenario — proves it's not canned.
3. Tab 2 for one sentence: "every model here beat, tied, or honestly lost to a stated baseline — we don't hide the losses."
4. Tab 6, corrupted scenario: "and it knows when to refuse."